

DataLab 实验报告

姓名：俞凡 学号：25301980234

实验仓库：Chineseyf/DataLab

完整实现见同目录的 bits.c，课程测试的原始输出见 verification.txt。

一、实验内容与环境

本次实验完成 bits.c 中的 19 个函数，主要练习补码、位掩码、移位、溢出判断和单精度浮点数舍入。实验环境为 VMware 中的 Ubuntu，通过 Windows 的 VS Code Remote SSH 编辑，使用仓库提供的 GCC 构建脚本和检查程序验证代码。

整数题只能使用指定的位运算和加法，常量限制为 0 到 255，不能写 if、循环、减法和类型转换。第二题限制更严格，只允许使用 ~ 和 &。局部变量使用 int，赋值本身不计入运算次数。

P15 到 P17 是浮点题，可以用 int、unsigned、条件判断和循环，但仍不能直接使用浮点类型或浮点运算。以下代码统一采用简短变量名，如 now、a、b、dig、mask，代码外说明变量作用。

本实验按 32 位补码模型解释整数运算：算术右移补符号位，超出 32 位的高位丢弃，移位量必须为 0 到 31。文中的 0xffffffff 等大常量可用于解释位模式；整数题实现中仍只使用符合限制的小常量来构造它们。

二、逐题实现与说明

P1 构造符号位掩码

返回一个只有最高位为 1、其他位全为 0 的 32 位整数。

```
int signMask(void) {  
    return (1 << 31);  
}
```

数字 1 的最低位为 1，把它向左移动 31 位，就能把这个 1 放到最高位。这里使用的常量只有 1 和 31，均在允许范围内。

```
1:          00000000 00000000 00000000 00000001  
1 << 31:    10000000 00000000 00000000 00000000
```

结果是 0x80000000。按本实验的 32 位有符号补码解释，它表示 -2147483648，而不是正的 2147483648。生成符号位的写法依赖本实验规定的位运算模型。

P2 只用取反和按位与实现异或

实现 $x \oplus y$ ，但只能使用 $\sim$ 和 $\&$ ，最多 8 次运算。

```
int bitXor(int x, int y) {  
    return ~(~x & y) & ~(x & ~y);  
}
```

异或在两位不同时为 1。 $\sim x \& y$ 选出“x 为 0、y 为 1”的位置， $x \& \sim y$ 选出“x 为 1、y 为 0”的位置，再把这两部分合起来。

原本可以用 $|$ 合并，但本题不允许用它。根据德摩根律， $a | b$ 等价于 $\sim(\sim a \& \sim b)$ ，代入两部分即可得到代码。

以 $x = 4$ 、 $y = 5$ 为例，只画最低 4 位：

```
x:      0100  
y:      0101  
~x & y:  0001  
x & ~y:  0000  
合并:    0001
```

所以返回 1。如果两个参数相同，每一位都相同，结果为 0。

P3 保留负数的相反数

当 x 为负数时返回其相反数，否则返回 0。

```
int negativePart(int x){  
    int now = (x >> 31);  
    return now & (~x + 1);  
}
```

有符号整数算术右移时，左边补符号位。因此 $x \gg 31$ 在 x 为负数时得到全 1，在 x 非负时得到全 0。全 1 与一个数按位与会保留这个数，全 0 则把结果清零，可以用来代替条件判断。

补码取负使用 $\sim x + 1$ 。例如 $x = -10$ 时，`now` 是全 1，保留取负后的 10； $x = 5$ 时，`now` 是全 0，结果为 0。

边界是 $x = -2147483648$ ：它的数学相反数超出 `int` 的范围，按本实验保留低 32 位的规则，结果仍是 `0x80000000`。

P4 在整数内部复制字节

把 x 的第 src 个字节复制到第 dst 个字节，其余字节不变。字节从右到左编号为 0 到 3，复制不是交换。

```
int copyByteWithin(int x, int src, int dst) {
    int a = src << 3;
    int b = dst << 3;
    int dig = (x >> a) & 0xff;
    int mask = 0xff << b;
    return (x & ~mask) | (dig << b);
}
```

每个字节占 8 位，所以 $src \ll 3$ 和 $dst \ll 3$ 分别得到源字节、目标字节的起始位置。先把源字节右移到最低位，用 $\& 0xff$ 取出它，再把目标位置清零，最后将源字节移过去并合并。

以 `copyByteWithin(0x11223344, 0, 2)` 为例：

```
字节编号:      3   2   1   0
原来的 x:      11  22  33  44
清空第 2 字节: 11  00  33  44
放入 44:        11  44  33  44
```

结果是 `0x11443344`。 dig 在清空目标位置之前就已从原来的 x 中取出，因此 src 与 dst 相同时也能正确保留原值。

P5 逻辑右移

把 x 右移 n 位，左边空出来的位置全部补 0， n 的范围为 0 到 31。

```
int logicalShift(int x, int n) {
    int now = x >> n;
    int mask = ~(((1 << 31) >> n) << 1);
    return now & mask;
}
```

先用 $x \gg n$ 完成算术右移，再构造高 n 位为 0、其余位为 1 的 $mask$ ，把左边补出来的符号位清掉。

构造 $mask$ 时， $1 \ll 31$ 只有最高位为 1；算术右移 n 位后，高 $n+1$ 位为 1；再左移 1 位，高 n 位为 1，其余位为 0。最后取反，就得到需要的掩码。

以 `logicalShift(0x87654321, 4)` 为例：

```
now:    0xf8765432
mask:    0x0fffffff
结果:    0x08765432
```

不能直接写成 $\sim((1 \ll 31) \gg (n + \sim 0))$ ：当 n 为 0 时，它会右移 -1 位，违反移位量必须在 0 到 31 之间的规则。当前写法在 n 为 0 时得到全 1 掩码，返回原值；在 n 为 31 时只保留最低 1 位。

P6 交换每个字节内的两组四位

将每个字节的高 4 位和低 4 位交换，字节之间的位置不变。

```
int swapNibblePairs(int x) {
    int mask = 0x0f | (0x0f << 8);
    int a, b;
    mask = mask | (mask << 16);
    a = x & mask;
    b = (x >> 4) & mask;
    return (a << 4) | b;
}
```

先用小常量拼出 0x0f0f0f0f，它在每个字节的低 4 位为 1。a 保存各字节原来的低 4 位，b 保存右移后的高 4 位，再把 a 左移 4 位，与 b 合并。

例如一个字节 0xab 的变化如下：

```
原来:      1010 1011
a:         0000 1011
b:         0000 1010
a << 4:    1011 0000
合并:      1011 1010, 即 0xba
```

因此 0x12345678 会变成 0x21436587。x >> 4 对负数可能补 1，但随后的掩码会去掉这些多余位。

P7 定位第二个零位

从最低位向最高位数，找到第二个为 0 的位置，返回只有该位置为 1 的掩码；不足两个 0 时返回 0。

```
int secondLowestZeroBit(int x) {
    int now = x | (x + 1);
    return ~now & (now + 1);
}
```

x + 1 会把最低端连续的 1 清零，并把遇到的第一个 0 变成 1。再与原来的 x 按位或，原有的 1 全部保留，因此 x | (x + 1) 只把最低的 0 改成了 1。

这样，原来的第二个 0 就变成 now 中最低的 0。~now & (now + 1) 能提取它：该位置在两部分中都为 1；更低位在 now+1 中被清零；更高位在加一时未改变，与原值的取反相与后为 0。

以 x = 0xffffffffa 为例，高位均为 1，下表只画最低 8 位：

```
x:          11111010
now:        11111011
~now:       00000100
now + 1:    11111100
结果:       00000100, 即 4
```

当 x 为全 1，或只有一个 0 时，now 都会变成全 1，最终结果为 0。

P8 计算奇校验位

x 的二进制中有偶数个 1 时返回 1，有奇数个 1 时返回 0，使加入校验位后的 1 的总数为奇数。

```
int oddParity(int x) {
    int now = x ^ (x >> 16);
    now = now ^ (now >> 8);
    now = now ^ (now >> 4);
    now = now ^ (now >> 2);
    now = now ^ (now >> 1);
    return !(now & 1);
}
```

异或只保留奇偶信息：两个 1 异或为 0，一个 1 和一个 0 异或为 1。依次右移 16、8、4、2、1 位并异或，最后最低位就是原来全部 32 位的异或结果。每一步都将更远位置的信息合并进来，最终最低位覆盖了所有原始位。

`now & 1` 在原来有奇数个 1 时为 1，偶数个 1 时为 0，题目要求相反，所以最后再用 `!` 翻转。

例如 x 为 5，二进制是 0101。前面右移 16、8、4 位均为 0，now 仍为 5；随后只画最低 4 位：

```
右移 2 位并异或：0101 ^ 0001 = 0100
右移 1 位并异或：0100 ^ 0010 = 0110
取最低位：      0110 & 0001 = 0000
逻辑非：        !0 = 1
```

结果为 1，与两个 1 属于偶数相符。这里没有计算“1 一共有几个”，只计算了个数的奇偶性；负数也按完整的 32 位位模式处理。

P9 循环右移

把 x 循环右移 n 位，右边移出去的位接回左边。n 非负，实际移动位数按 32 取模。

```
int rotateRightBits(int x, int n) {
    int mask, a, b;
    n = n & 31;
    mask = ~(((1 << 31) >> n) << 1);
    a = (x >> n) & mask;
    b = x << ((~n + 1) & 31);
    return a | b;
}
```

先用 `n & 31` 得到 n 除以 32 的余数。a 使用第五题的掩码方法得到逻辑右移部分；b 把原来低 n 位移到左边，最后按位或。

例如 `rotateRightBits(0x12345678, 8)`：

```
a:    0x00123456
b:    0x78000000
合并：0x78123456
```

通常左移量是 32-n，但 n 为 0 时不能左移 32 位。`(~n + 1) & 31` 把这个移位量限制在 0 到 31：n 为 0 时 a、b 都等于 x，两者按位或仍为 x。n 为 32、64 等值时也返回原值。

P10 舍入到二的幂的倍数

把非负整数 x 舍入到最近的 2^n 的倍数；恰好在中间时，选择除以 2^n 后商为偶数的那个倍数。输入满足 x 不超过 $0x3ffffff$ ， n 为 1 到 16。

```
int roundEvenPow2(int x, int n) {
    int a = 1 << (n + ~0);
    int b = (x >> n) & 1;
    int now = x + a + ~0 + b;
    return (now >> n) << n;
}
```

a 是相邻两个倍数间距的一半， b 是原来整数商的最低位。先加上 $a - 1 + b$ ，再右移 n 位取整、左移 n 位恢复为倍数。其中减 1 用 $+ ~0$ 表示。

当余数小于或大于半程时，这个偏移量分别让结果落到较小或较大的倍数；只有恰好半程时， b 决定是否向上，保证最终的商为偶数。

例如 n 为 2，间距为 4、半程为 2：

输入 x	原来的整数商	加上的偏移量	结果
10	2, 偶数	$2-1+0 = 1$	8
14	3, 奇数	$2-1+1 = 2$	16
11	2, 偶数	1	12

所以 10 与 14 虽然都在中点，舍入方向不同。给定的输入上限保证加偏移量时不会发生有符号溢出。

P11 求偏向第一个参数的中点

求 x 、 y 的数学中点，避免 $x+y$ 溢出；中点不是整数时，选择更靠近第一个参数 x 的相邻整数。

```
int midpointTowardFirst(int x, int y) {
    int a = x ^ y;
    int b = (x & y) + (a >> 1);
    int now = x + (~y + 1);
    int mask = ((a >> 31) & ~(x >> 31)) | (~(a >> 31) & ~(now >> 31));
    return b + (a & mask & 1);
}
```

$(x \& y) + ((x \wedge y) \gg 1)$ 得到向下取整的平均值。可以把 $x+y$ 理解为 $2 \times (x \& y) + (x \wedge y)$ ，先对异或部分右移，就不需要直接计算可能溢出的 $x+y$ 。

a 的最低位表示 x 、 y 的奇偶性是否不同，也就是中点是否在两个整数之间。若需要舍入且 x 大于 y ，就给向下取整的结果加 1；否则保持原值。

$mask$ 用来判断 x 是否不小于 y 。两数异号时直接看 x 的符号；同号时再看 $x-y$ 的符号，避免用溢出的差值比较异号整数。

例如 (4, 7) 的数学中点为 5.5，向下取整先得 5，较靠近 $x=4$ ，所以返回 5；(7, 4) 同样先得 5，但需要靠近 $x=7$ ，所以加 1，返回 6。(INT_MAX, INT_MAX) 则能直接得到 INT_MAX。

P12 判断是否位于两端点之间

当 x 位于由 a 、 b 确定的闭区间内时返回 1，否则返回 0。 a 、 b 不要求按大小排列。

```
int isBetweenEitherOrder(int x, int a, int b) {
    int p = x >> 31;
    int q = a >> 31;
    int r = b >> 31;
    int u = p ^ q;
    int v = p ^ r;
    int now = (u & p) | (~u & ((x + ~a + 1) >> 31));
    int mask = (v & p) | (~v & ((x + ~b + 1) >> 31));
    return ((now ^ mask) & 1) | !(x ^ a) | !(x ^ b);
}
```

now 表示 $x < a$ ， $mask$ 表示 $x < b$ ，成立时均为全 1，否则为全 0。如果 x 只小于其中一个端点，它就在两个端点之间；如果两个判断同真或同假，则通常在区间外。再单独判断 x 是否等于任一端点，就能得到包含端点的结果。

比较时分别处理符号： x 与端点异号，负数一定更小；两者同号，差值不会溢出，再用差值的符号判断。最后 $!(x \hat{=} a)$ 和 $!(x \hat{=} b)$ 用来判断相等。

例如 $x=5$ 、 $a=8$ 、 $b=3$ ， $x < a$ 为真、 $x < b$ 为假，两者异或后为 1，因此返回 1。若 $x=8$ ，则两个“小于”均为假，但 x 等于 a ，仍返回 1。若 $a=b$ ，则只有 x 与它们相等时返回 1。

P13 乘五并在溢出时饱和

计算 $x \times 5$ ；超过 int 上界时返回 INT_MAX ，低于下界时返回 INT_MIN 。

```
int mul5Sat(int x) {
    int a = x + x;
    int b = a + a;
    int now = b + x;
    int mask = ((x ^ a) | (x ^ b) | (x ^ now)) >> 31;
    int dig = (1 << 31) ^ ~(x >> 31);
    return (now & ~mask) | (dig & mask);
}
```

先计算 $a=2x$ 、 $b=4x$ ，再得到 $now=5x$ 。若某一步首次溢出，其符号会与原来的 x 不同，所以分别比较 a 、 b 、 now 与 x 的符号，合成溢出掩码。

只检查最终结果的符号不够，因为溢出后可能绕回与 x 同号的结果。检查中间倍增过程，可以捕捉第一次越界。

dig 根据 x 的符号构造饱和值： x 为非负时是 $0x7fffffff$ ， x 为负时是 $0x80000000$ 。 $mask$ 为全 0 时保留 now ，为全 1 时改用 dig 。

例如 $x=7$ 时返回 35； $x=0x40000000$ 时，数学结果为 5368709120，超过 INT_MAX ，返回 $0x7fffffff$ 。 $x=-500000000$ 时数学结果低于 INT_MIN ，返回 $0x80000000$ 。

P14 判断三个数的和是否溢出

判断精确的数学和 $x+y+z$ ：大于 `INT_MAX` 返回 1，小于 `INT_MIN` 返回 -1，否则返回 0。

```
int classifyAdd3(int x, int y, int z) {
    int a = (x >> 2) + (y >> 2) + (z >> 2);
    int b = (x & 3) + (y & 3) + (z & 3);
    int now = (a + (b >> 2)) >> 29;
    int mask = now >> 31;
    int dig = !(now ^ mask);
    return (dig ^ mask) + (mask & 1);
}
```

直接把三个 `int` 相加会丢失溢出信息，因此先把每个数拆成“除以 4 向下取整的商”和“0 到 3 的余数”。对正负数，都有 $x = 4 \times (x \gg 2) + (x \& 3)$ 。

a 是三个商之和， b 是三个余数之和。 a 的范围是 -1610612736 到 1610612733， b 的范围是 0 到 9，都能用 `int` 保存。 $a + (b \gg 2)$ 就是总和除以 4 后向下取整的值。

原和位于 `int` 范围内，等价于这个商位于 -536870912 到 536870911，即 -2^{29} 到 $2^{29}-1$ 。因此再右移 29 位：范围内只能得到 0 或 -1，超出范围则得到其他值。

`mask` 保存 `now` 的符号扩展。`!(now ^ mask)` 判断是否越界，最后将这个 0 或 1 按符号变成 0、1 或 -1。

例如 (`INT_MAX`, 1, 0) 的商为 536870912，再右移 29 位得到 1，返回 1；(`INT_MIN`, -1, 0) 得到 -2，返回 -1。(`INT_MAX`, `INT_MAX`, `INT_MIN`) 的精确和为 2147483646，仍在范围内，返回 0，不能因为前两个数相加溢出就提前下结论。

单精度浮点编码

P15 到 P17 操作的是下面这 32 位编码，而不是直接对 `float` 变量运算。

31	30	23 22	0
符号位	阶码 E (8 位)	尾数字段 F (23 位)	

普通数的值是 $(-1)^{\text{sign}} \times (1 + F / 2^{23}) \times 2^{(E-127)}$ ，所以要在尾数字段前补上隐含的 1，形成 24 位有效数。 E 为 0 时没有这个隐含的 1，数值为 $(-1)^{\text{sign}} \times (F / 2^{23}) \times 2^{(-126)}$ ，用于表示零和非常接近零的数。

E 为 255 时， F 为 0 表示无穷大， F 非零表示 NaN。就近取偶数舍入的意思是：先选择距离最近的可表示结果；恰好位于中点时，选择保留部分最低位为 0 的结果。

P15 浮点数乘以一点五

输入和输出都是单精度浮点数的 32 位编码，计算 $f \times 3/2$ ，采用就近舍入、恰好中点取偶数。NaN 原样返回，并保留正负零的符号。

```
unsigned floatScaleThreeHalves(unsigned uf) {
    unsigned sign = uf & 0x80000000;
    unsigned a = (uf >> 23) & 0xff;
    unsigned b = uf & 0x7fffff;
    unsigned now, mask, half;
    int n = 1;
    if (a == 0xff) return uf;
    if (a) b = b | 0x800000;
    else a = 1;
    now = b + (b << 1);
    if (now >= 0x2000000) {
        n = 2;
        a = a + 1;
    }
    mask = (1 << n) - 1;
    b = now & mask;
    now = now >> n;
    half = 1 << (n - 1);
    if (b > half || (b == half && (now & 1))) now = now + 1;
    if (now == 0x1000000) {
        now = now >> 1;
        a = a + 1;
    }
    if (a >= 255) return sign | 0x7f800000;
    if (now < 0x800000) a = 0;
    return sign | (a << 23) | (now & 0x7fffff);
}
```

sign 保存符号，a 保存阶码，b 保存尾数字段。阶码为 255 时是无穷大或 NaN，直接返回原编码；普通数要补上省略的最高有效位，非规格化数不补这个 1，但计算时按有效阶码 1 处理。

先把有效数乘 3。乘以 1.5 还需要除以 2，所以通常右移 1 位；若乘 3 后有效数太大，就右移 2 位并将阶码加 1。丢弃的低位保存在 b 中，half 是半程：大于半程时进位，恰好半程且保留部分为奇数时也进位。

舍入可能再次产生进位，需要再调整有效数和阶码。阶码达到 255 时返回带原符号的无穷大；有效数仍不足规格化范围时，将阶码设回 0。

例如 0x3f800000 表示 1.0，返回 0x3fc00000，表示 1.5。最小正非规格化数 0x00000001 乘以 1.5 后，正好在编码 1 和 2 对应的值中间，按偶数规则返回 0x00000002。0x80000000 表示负零，返回值仍为负零。

P16 把浮点数舍入到整数值

将输入浮点数舍入到最近整数，中点取偶数，但返回值仍是浮点数编码。NaN、无穷大原样返回，舍入成零时保留输入符号。

```
unsigned floatRoundEven(unsigned uf) {
    unsigned sign = uf & 0x80000000;
    unsigned a = (uf >> 23) & 0xff;
    unsigned now, mask, b, half;
    int n;
    if (a >= 150) return uf;
    if (a < 126) return sign;
    if (a == 126) {
        if (!(uf & 0x7fffff)) return sign;
        return sign | 0x3f800000;
    }
    n = 150 - a;
    mask = (1 << n) - 1;
    b = uf & mask;
    now = uf & ~mask;
    half = 1 << (n - 1);
    if (b > half || (b == half && ((now >> n) & 1))) now = now + (1 << n);
    return now;
}
```

单精度有 24 位有效精度，阶码达到 150，即实际指数至少为 23 时，有限数已经没有小数部分，可以直接返回；这个分支也保留阶码为 255 的无穷大和 NaN。

阶码小于 126 时，绝对值小于 0.5，返回带符号的零。阶码等于 126 时，恰好 0.5 舍入成零，超过 0.5 则舍入成带符号的 1。

其余情况用 $n = 150 - a$ 算出编码中需要清除的小数位。now 清除这些位，b 保存被清掉的部分；与 half 比较后决定是否给整数部分加 1。恰好半程时，还要看整数部分最低位是否为 1。

浮点输入值	舍入结果值	返回编码
1.5	2.0	0x40000000
2.5	2.0	0x40000000
-0.5	-0.0	0x80000000

这里返回的 0x40000000 是 2.0 的位编码，不是整数 2。负数使用相同的绝对值舍入规则，符号位保留。

P17 将整数转换为单精度编码

返回将整数 x 转为单精度浮点数后的位编码，不能直接使用 `float` 或类型转换。

```
unsigned float_i2f(int x) {
    unsigned sign = x & 0x80000000;
    unsigned now = x;
    unsigned a, b, mask, half;
    int n = 0;
    if (!x) return 0;
    if (x < 0) now = ~now + 1;
    a = now;
    while (a >> 1) {
        a = a >> 1;
        n = n + 1;
    }
    if (n <= 23) return sign | ((n + 127) << 23) | ((now << (23 - n)) & 0x7fffff);
    a = n - 23;
    mask = (1 << a) - 1;
    b = now & mask;
    half = 1 << (a - 1);
    now = now >> a;
    if (b > half || (b == half && (now & 1))) now = now + 1;
    if (now == 0x1000000) {
        now = now >> 1;
        n = n + 1;
    }
    return sign | ((n + 127) << 23) | (now & 0x7fffff);
}
```

先保存符号，再用 `unsigned` 保存绝对值。负数在 `unsigned` 中取反加一，这样即使 x 是 `INT_MIN`，也能保存其绝对值 2147483648。

循环找到最高 1 的位置 n ，实际指数就是 n ，阶码为 $n+127$ 。若 n 不超过 23，把有效数左移对齐即可，转换是精确的。否则右移丢弃低位，并记录被丢弃的值与半程值，按就近取偶数规则决定是否加 1。

如果舍入后有效数从 24 位变成 25 位，就再右移一次并把指数加 1，最后拼回符号、阶码和尾数字段。

例如 5 的二进制为 101，写成 1.01×2^2 ，阶码为 129，尾数字段对应 .01，返回 0x40a00000。16777217 恰好位于两个可表示的浮点值 16777216 和 16777218 之间，取偶数后返回 16777216 的编码 0x4b800000。

P18 统计一的个数

返回 x 的 32 位二进制中 1 的总个数。

```
int bitCount(int x) {
    int a = 0x55 | (0x55 << 8);
    int b = 0x33 | (0x33 << 8);
    int mask = 0x0f | (0x0f << 8);
    int now;
    a = a | (a << 16);
    b = b | (b << 16);
    mask = mask | (mask << 16);
    now = (x & a) + ((x >> 1) & a);
    now = (now & b) + ((now >> 2) & b);
    now = (now + (now >> 4)) & mask;
    now = now + (now >> 8);
    now = now + (now >> 16);
    return now & 0x3f;
}
```

先用 0x55555555 把每两位分开相加，每组得到一个 0 到 2 的计数；再用 0x33333333 把相邻的两组加起来，每 4 位保存一个 0 到 4 的计数。

接着把相邻两组 4 位计数相加，并用 0x0f0f0f0f 保留每字节的结果。最后通过右移 8、16 位相加，将各字节的计数汇总到最低位。总数最多 32，只需最低 6 位，因此使用 & 0x3f。

以 x=7 为例，只画最低 8 位：

原始位：	00000111
每两位计数：	00 00 01 10
每四位计数：	0000 0011
整个字节计数：	00000011，即 3

因此返回 3。x=-1 时 32 位全部为 1，返回 32；x=0 时返回 0。与第八题不同，这里确实计算完整个数，所以使用加法汇总，不能只用异或。

P19 反转全部三十二位

将 x 的 32 个二进制位前后反转：原来的第 0 位移到第 31 位，第 1 位移到第 30 位，以此类推。

```
int bitReverse(int x) {
    int mask = 0xff | (0xff << 8);
    int now = (x << 16) | ((x >> 16) & mask);
    mask = mask ^ (mask << 8);
    now = ((now >> 8) & mask) | ((now & mask) << 8);
    mask = mask ^ (mask << 4);
    now = ((now >> 4) & mask) | ((now & mask) << 4);
    mask = mask ^ (mask << 2);
    now = ((now >> 2) & mask) | ((now & mask) << 2);
    mask = mask ^ (mask << 1);
    return ((now >> 1) & mask) | ((now & mask) << 1);
}
```

分五步交换：先交换高低 16 位，再在各组内部交换 8 位、4 位、2 位，最后交换相邻的单个位。每一步都用掩码清除算术右移补进来的符号位。

为控制操作次数，没有分别从头构造所有掩码。初始 mask 为 0x0000ffff，随后用 $\text{mask} \wedge (\text{mask} \ll n)$ 依次生成更细的分组掩码：

交换大小	使用的 mask	作用
16 位	0x0000ffff	保留右移后的低 16 位
8 位	0x00ff00ff	每 16 位中保留低 8 位
4 位	0x0f0f0f0f	每字节保留低 4 位
2 位	0x33333333	每 4 位保留低 2 位
1 位	0x55555555	每 2 位保留低 1 位

例如 0x80000004 只有第 31 位和第 2 位为 1。反转后，它们分别位于第 0 位和第 29 位，因此结果为 0x20000001。本实现使用 34 次运算，恰好达到本题上限。

三、验证过程与结果

配套代码已在 Ubuntu 的独立副本中执行课程提供的 `./test.sh`。该脚本会重新构建程序、检查运算规则并运行全部正确性测试。19 题全部通过，btest 总分为 110/110；这是本地测试结果，不代表 GitHub 自动评分或 E-Learning 提交已完成。

```
./test.sh
```

真实输出的关键行如下，完整记录见 `verification.txt`：

```
All 19 functions passed operator checks.
Total points: 110/110
all checks passed: 110/110
```

题目	实测运算次数/上限	btest 得分与错误数
P1 signMask	1/2	1/1, Errors = 0
P2 bitXor	8/8	2/2, Errors = 0
P3 negativePart	4/6	3/3, Errors = 0
P4 copyByteWithin	9/12	4/4, Errors = 0
P5 logicalShift	6/20	4/4, Errors = 0
P6 swapNibblePairs	9/18	4/4, Errors = 0
P7 secondLowestZeroBit	5/8	4/4, Errors = 0
P8 oddParity	12/56	5/5, Errors = 0
P9 rotateRightBits	12/16	5/5, Errors = 0
P10 roundEvenPow2	11/24	5/5, Errors = 0
P11 midpointTowardFirst	20/32	5/5, Errors = 0
P12 isBetweenEitherOrder	29/48	7/7, Errors = 0
P13 mul5Sat	17/30	7/7, Errors = 0
P14 classifyAdd3	20/52	7/7, Errors = 0
P15 floatScaleThreeHalves	32/60	7/7, Errors = 0
P16 floatRoundEven	25/65	10/10, Errors = 0
P17 float_i2f	37/40	10/10, Errors = 0
P18 bitCount	28/40	10/10, Errors = 0
P19 bitReverse	34/34	10/10, Errors = 0

四、关键问题与理解

取反和逻辑非的区别

`~` 对全部 32 位逐位取反，`!` 只判断输入是不是零。例如 `~0` 得到全 1，即 -1；`!0` 得到 1；对任何非零数使用 `!` 都得到 0。`!!x` 则把任意整数转换为 0 或 1，因此可以用来整理判断结果。

用掩码代替条件选择

一个数与全 1 按位与会保持原样，与全 0 按位与则变为 0。因此可以先把条件变成全 0 或全 1 的 mask，再用按位与、按位或选出结果。第三题保留负数的相反数、第十三题在普通结果和饱和值之间选择，都用了这个方法。

边界不能只靠普通例子检查

第五题的 `n=0` 会暴露负数移位量的问题，第九题也必须避免移位 32 位。涉及符号比较时，要考虑异号减法可能溢出；涉及取负时，要考虑 `INT_MIN` 没有可表示的正数相反数。浮点题则还要区分正负零、非规格化数、NaN、无穷大和恰好半程的舍入。

五、实验小结

本次实现中，掩码负责选位，移位负责改变位置，按位与和按位或负责清除、保留与合并。掌握这些操作后，就可以在没有分支和循环的限制下完成字节复制、逻辑右移和条件选择。